

PHP Orientado a Objetos

Lista de Exercícios

Curso Técnico em Informática | 2 horas de aula

Tópicos abordados

Classes, objetos, encapsulamento, construtores, getters e setters

Estrutura da lista

3 questões teóricas · 4 encontre o erro · 8 práticas

ATENÇÃO: O professor estará ausente. Leia os enunciados com calma, use as dicas e consulte o colega antes de desistir.

Parte 1 — Teoria: Encapsulamento

Responda às questões a seguir por escrito. Seja claro e use exemplos quando possível.

1

O que é encapsulamento?

Teoria

Responda com suas próprias palavras:

O que significa encapsular dados em uma classe?

Resposta: significa proteger dados de um objeto definindo como e de onde eles podem ser acessados ou modificados.

Qual é a diferença entre um atributo public, protected e private?

public permite que o atributo ou método seja acessado de qualquer lugar do programa.

private permite acesso apenas dentro da própria classe.

protected permite acesso dentro da classe e também pelas classes que herdam dela.

Por que não é boa prática deixar todos os atributos como public?

Porque atributos public podem ser modificados livremente, o que reduz a proteção e o controle dos dados da classe.

Dica

Pense em encapsulamento como uma capsula protetora: os dados ficam dentro da classe e so sao acessados por metodos controlados. private = so a propria classe acessa; protected = a classe e suas filhas; public = qualquer lugar do codigo.

2

Para que servem os métodos get e set?

Teoria

Responda:

Por que usamos getAtributo() ao invés de acessar \$objeto->atributo diretamente?

Usamos getAtributo() para acessar os dados de forma controlada e segura, sem modificar diretamente o atributo do objeto. Isso ajuda no encapsulamento, protegendo os dados da classe

O que um método `set` pode fazer além de simplesmente atribuir um valor?
`set` pode fazer validação dos dados antes de atribuir um valor ao atributo, e também pode executar outras ações como notificar mudanças, recalcular valores ou registrar logs

Dê um exemplo prático de validação dentro de um `set`.
O método `setIdade()` aceita apenas idades dentro de uma faixa lógica (1 a 119), evitando valores inválidos como idade negativa.

Dica

Um `setIdade($valor)` pode verificar se `$valor > 0` antes de salvar. Isso protege o objeto de receber dados inválidos.

3 Verdadeiro ou Falso — encapsulamento

Teoria

Classifique cada afirmação como Verdadeiro (V) ou Falso (F) e justifique:

"Um atributo `private` pode ser lido diretamente fora da classe." Falso atributos `private` só podem ser acessados dentro da própria classe.

"Getters e setters só fazem sentido quando os atributos são `private`." Verdadeiro eles são usados principalmente para controlar o acesso a atributos privados.

"Encapsulamento aumenta a segurança e a manutenção do código." Verdadeiro encapsulamento protege os dados e evita manipulação incorreta, tornando o código mais seguro e organizado.

"É possível ter um getter sem ter um setter para o mesmo atributo." Verdadeiro um atributo pode permitir apenas leitura, sem permitir alteração do valor.

Dica

Lembre-se: atributos `private` só são acessíveis dentro da própria classe. Fora dela, qualquer acesso direto gera erro fatal em PHP.

Parte 2 — Encontre o Erro

Cada questão abaixo contém um código PHP com um erro. Você deve:

Identificar a linha ou trecho com erro
`echo $p->nome;`

Explicar por que é um erro

O atributo `$nome` foi declarado como `private`, então ele não pode ser acessado diretamente fora da classe. Apenas métodos da própria classe podem acessar atributos privados.

Apresentar o código corrigido com comentários

```
<?php
class Produto {
    private $nome;
    private $preco;
    public function __construct($nome, $preco) {
        $this->nome = $nome;
        $this->preco = $preco;
    }
    public function getNome() {
        return $this->nome;
    }
}
$p = new Produto('Teclado', 150.00);
```

// Forma correta de acessar o atributo private

```
echo $p->getNome();
```

4

Erro: acesso a atributo privado

Encontre o erro

```
<?php
class Produto {
    private $nome;
    private $preco;

    public function __construct($nome, $preco) {
        $this->nome = $nome;
        $this->preco = $preco;
    }
}

$p = new Produto('Teclado', 150.00);
echo $p->nome; // ESTA LINHA TEM ERRO
?>
```

Dica

O atributo \$nome é private. Acesso direto fora da classe é proibido em PHP. Crie um método getNome() e chame \$p->getNome().

O atributo \$nome é private, então não pode ser acessado diretamente fora da classe.

Forma correta de acessar atributo private

```
echo $p->getNome();
```

```

<?php

class Produto {

    private $nome;
    private $preco;
    public function __construct($nome, $preco) {
        $this->nome = $nome;
        $this->preco = $preco;
    }
    public function getNome() {
        return $this->nome;
    }
}

$p = new Produto('Teclado', 150.00);

// Forma correta de acessar atributo private
echo $p->getNome();

?>

```

5

Erro: uso de variável sem \$this

Encontre o erro

```

<?php
class Aluno {
    private $nome;
    private $nota;

    public function __construct($nome, $nota) {
        $this->nome = $nome;
        $this->nota = $nota;
    }

    public function exibir() {
        echo 'Aluno: ' . $nome . ' - Nota: ' . $nota; // ERRO AQUI
    }
}

$a = new Aluno('Carlos', 8.5);
$a->exibir();
?>

```

Dica

Dentro dos métodos da classe, os atributos devem ser acessados com `$this->nome`, não simplesmente `$nome` (que seria uma variável local inexistente).

Os atributos **\$nome** e **\$nota** foram declarados na classe como atributos privados:

`$this` significa este objeto

Como esses atributos pertencem ao objeto, eles não podem ser acessados apenas pelos nomes `$nome` e `$nota` dentro dos métodos da classe. Ao escrever dessa forma, o PHP procura por variáveis locais chamadas `$nome` e `$nota` dentro do método `exibir()`. Como essas variáveis não existem, ocorre um erro de variável indefinida.

Para acessar atributos do próprio objeto, é necessário utilizar **`$this->`**, pois a palavra **`$this`** representa o objeto atual que está executando o método. Portanto, a linha incorreta:

```
echo 'Aluno: ' . $nome . ' - Nota: ' . $nota;
```

código corrigido

```
<?php
```

```
class Aluno {  
    private $nome;  
    private $nota;  
    public function __construct($nome, $nota) {  
        $this->nome = $nome;  
        $this->nota = $nota;  
    }  
    public function exibir() {  
        echo 'Aluno: ' . $this->nome . ' - Nota: ' . $this->nota;  
    }  
}
```

```
}
```

```
$a = new Aluno('Carlos', 8.5);
```

```
$a->exibir();
```

```
?>
```

6

Erro: setter não bloqueia valor inválido

Encontre o erro

```
<?php
class Pessoa {
    private $idade;

    public function setIdade($idade) {
        if ($idade < 0) {
            echo 'Idade invalida!';
        }
        $this->idade = $idade; // problema: atribui mesmo se invalido
    }

    public function getIdade() {
        return $this->idade;
    }
}

$p = new Pessoa();
$p->setIdade(-5);
echo $p->getIdade(); // imprime -5 !
?>
```

Dica

O if avisa o erro mas não impede a atribuição. Use return após a mensagem de erro, ou coloque a atribuição dentro de um else, para que valores inválidos não sejam salvos.

```
<?php
```

```
class Pessoa {
    private $idade;
```

```
    public function setIdade($idade) {
        if ($idade < 0) {
            echo 'Idade invalida!';
            return;
        }
        $this->idade = $idade;
    }
}
```

```

    public function getIdade() {
        return $this->idade;
    }
}

```

```

$p = new Pessoa();
$p->setIdade(-5);
echo $p->getIdade();
?>

```

setter não bloqueia o valor inválido. Mesmo mostrando a mensagem “Idade inválida!”, o código continua executando e salva -5 na variável idade. O correto seria interromper o método usando return para impedir a atribuição do valor inválido.

7

Erro: instanciação sem new

Encontre o erro

```

<?php

class Carro {
    private $modelo;

    public function __construct($modelo) {
        $this->modelo = $modelo;
    }

    public function getModelo() {
        return $this->modelo;
    }
}

$c = Carro('Fusca'); // ERRO NESTA LINHA
echo $c->getModelo();
?>

```

Dica

Para criar um objeto em PHP, é obrigatório usar a palavra-chave new: `$c = new Carro('Fusca');` Sem ela, o PHP trata `Carro()` como uma chamada de função inexistente.

Em PHP, para criar um objeto de uma classe, é obrigatório utilizar a palavra-chave new.

Sem o new, o PHP interpreta `Carro('Fusca')` como se fosse uma chamada de função chamada `Carro()`. Como essa função não existe, ocorre um erro.

Como a classe Carro possui um método construtor (___construct), o objeto deve ser criado corretamente usando :new Carro('Fusca')

ficaria assim o código corrigido

```
<?php
```

```
class Carro {  
    private $modelo;  
  
    public function __construct($modelo) {  
        $this->modelo = $modelo;  
    }  
  
    public function getModelo() {  
        return $this->modelo;  
    }  
}
```

```
$c = new Carro('Fusca')  
echo $c->getModelo();
```

```
?>
```

Parte 3 — Exercícios Práticos

Implemente as classes descritas abaixo. Teste cada uma criando objetos e chamando os métodos.

8

Classe Livro

Prático

Crie a classe Livro com os seguintes requisitos:

- Atributos privados: \$titulo, \$autor, \$paginas
- Construtor que recebe os três valores
- Getters e setters para todos os atributos
- Método exibir() que imprime todas as informações formatadas
- Instancie dois livros diferentes e chame exibir() em ambos

Dica

Estrutura: class Livro { private \$titulo; public function __construct(\$t,\$a,\$p){ \$this->titulo=\$t; ... } }. O método exibir() usa echo com os getters.


```
<?php
class Livro {
    private $titulo;
    private $autor;
    private $paginas;
    public function __construct($titulo, $autor, $paginas) {
        $this->titulo = $titulo;
        $this->autor = $autor;
        $this->paginas = $paginas;
    }
    // Getter do título
    public function getTitulo() {
        return $this->titulo;
    }
    // Setter do título
    public function setTitulo($titulo) {
        $this->titulo = $titulo;
    }
    // Getter do autor
    public function getAutor() {
        return $this->autor;
    }
    // Setter do autor
    public function setAutor($autor) {
        $this->autor = $autor;
    }
    // Getter do número de páginas
    public function getPaginas() {
        return $this->paginas;
    }
    // Setter do número de páginas
    public function setPaginas($paginas) {
        $this->paginas = $paginas;
    }
    // Método para exibir as informações do livro
    public function exibir() {
        // Mostra o título usando o getter
        echo "Título: " . $this->getTitulo() . "<br>";
        // Mostra o autor usando o getter
        echo "Autor: " . $this->getAutor() . "<br>";
        // Mostra a quantidade de páginas usando o getter
```

```

        echo "Páginas: " . $this->getPaginas() . "<br><br>";
    }
}
$livro1 = new Livro("Dom Casmurro", "Machado de Assis", 256);
$livro2 = new Livro("O Pequeno Príncipe", "Antoine de Saint-Exupéry", 96);
$livro1->exibir();
$livro2->exibir();
?>

```

Primeiro foi criada a classe Livro, contendo os atributos privados \$titulo, \$autor e \$paginas. Em seguida foi criado o método construtor `__construct()`, responsável por receber os valores desses atributos quando um objeto é criado.

Depois foram implementados os métodos getters e setters, que permitem acessar e alterar os atributos privados de forma segura.

Também foi criado o método `exibir()`, que mostra na tela todas as informações do livro utilizando os getters.

Por fim, foram instanciados dois objetos da classe Livro (`$livro1` e `$livro2`) com informações diferentes, e o método `exibir()` foi chamado em cada um deles para apresentar seus dados.

9 Classe ContaBancaria

Prático

Crie a classe ContaBancaria:

Atributos privados: \$titular, \$saldo
 Construtor recebe \$titular e inicia \$saldo com 0
 Método depositar(\$valor) — soma ao saldo (validar: valor positivo)
 Método sacar(\$valor) — subtrai do saldo (validar: saldo suficiente)
 Getters para \$saldo e \$titular
 Teste: crie uma conta, deposite R\$500, saque R\$200, exiba o saldo final

Dica

No metodo sacar, use: `if ($valor > $this->saldo) { echo 'Saldo insuficiente'; return; }` antes de subtrair.

```

<?php
class ContaBancaria {
    private $titular;
    private $saldo;
    // Construtor
    public function __construct($titular) {
        $this->titular = $titular;
    }
}

```

```

        $this->saldo = 0;
    }
    // Getters
    public function getTitular() {
        return $this->titular;
    }
    public function getSaldo() {
        return $this->saldo;
    }
    // Deposita um valor na conta
    public function depositar($valor) {
        if ($valor <= 0) {
            echo "O valor deve ser positivo.<br>";
            return;
        }

        $this->saldo += $valor;
    }

    // Realiza um saque
    public function sacar($valor) {
        if ($valor > $this->saldo) {
            echo "Saldo insuficiente.<br>";
            return;
        }

        $this->saldo -= $valor;
    }
}

// Criação da conta
$conta = new ContaBancaria("Luiza");
// Operações
$conta->depositar(500);
$conta->sacar(200);
// Exibe os dados finais
echo "Titular: " . $conta->getTitular() . "<br>";
echo "Saldo final: R$ " . $conta->getSaldo();
?>

```

Foi criada a classe ContaBancaria com os atributos privados \$titular e \$saldo. O construtor recebe o nome do titular e inicia o saldo com 0.

O método depositar() adiciona um valor ao saldo, desde que ele seja positivo. O método sacar() desconta um valor do saldo, mas antes verifica se há saldo suficiente.

Por fim, foi criada uma conta para Luiza, realizado um depósito de R\$500 e um saque de R\$200. O saldo final exibido é R\$300.

10 Classe Aluno com média

Prático

Crie a classe Aluno:

Atributos privados: \$nome, \$nota1, \$nota2
Construtor recebe os três valores
Getters e setters para todos (notas devem ser entre 0 e 10)
Método calcularMedia() que retorna a média das duas notas
Método situacao() que imprime 'Aprovado' (média ≥ 5) ou 'Reprovado'

Dica

calcularMedia() retorna $(\$this->nota1 + \$this->nota2) / 2$. Em situacao(), chame $\$this->calcularMedia()$ para evitar repetir o calculo.

```
<?php
class Aluno {
    private $nome;
    private $nota1;
    private $nota2;
    // Construtor
    public function __construct($nome, $nota1, $nota2) {
        $this->nome = $nome;
        $this->nota1 = $nota1;
        $this->nota2 = $nota2;
    }
    // Getters
    public function getNome() {
        return $this->nome;
    }
    public function getNota1() {
        return $this->nota1;
    }
    public function getNota2() {
        return $this->nota2;
    }
    // Setters
    public function setNome($nome) {
```

```

        $this->nome = $nome;
    }
    public function setNota1($nota1) {
        if ($nota1 >= 0 && $nota1 <= 10) {
            $this->nota1 = $nota1;
        }
    }

    public function setNota2($nota2) {
        if ($nota2 >= 0 && $nota2 <= 10) {
            $this->nota2 = $nota2;
        }
    }

    // Calcula a média
    public function calcularMedia() {
        return ($this->nota1 + $this->nota2) / 2;
    }

    // Mostra a situação do aluno
    public function situacao() {
        if ($this->calcularMedia() >= 5) {
            echo "Aprovado";
        } else {
            echo "Reprovado";
        }
    }
}

// Criação do aluno
$aluno = new Aluno("Luiza", 8, 6);
// Exibe os dados
echo "Nome: " . $aluno->getNome() . "<br>";
echo "Média: " . $aluno->calcularMedia() . "<br>";
echo "Situação: ";
$aluno->situacao();
?>

```

Foi criada a classe Aluno com os atributos privados \$nome, \$nota1 e \$nota2. O construtor recebe esses valores quando o objeto é criado.

Os métodos getters retornam os valores dos atributos e os setters permitem alterá-los. Nos setters das notas foi adicionada uma validação para aceitar apenas valores entre 0 e 10.

O método `calcularMedia()` retorna a média das duas notas. Já o método `situacao()` utiliza a média calculada para verificar se o aluno está Aprovado (média maior ou igual a 5) ou Reprovado (média menor que 5).

No teste foi criado um aluno com notas 8 e 6, resultando em média 7, portanto sua situação é Aprovado.

11 Classe Retangulo

Prático

Crie a classe Retangulo:

- Atributos privados: `$largura` e `$altura`
- Construtor recebe os dois valores (validar: devem ser positivos)
- Getters e setters para ambos
- Método `calcularArea()` que retorna `largura x altura`
- Método `calcularPerimetro()` que retorna `2 x (largura + altura)`
- Crie dois retângulos e compare qual tem maior área

Dica

Para comparar: `if ($r1->calcularArea() > $r2->calcularArea()) { ... }`. Os métodos são chamados no objeto com `->`.

12 Classe Funcionario com salário

Intermediário

Crie a classe Funcionario:

- Atributos privados: `$nome`, `$cargo`, `$salario`
- Construtor recebe os três valores
- Getters para todos os atributos
- Método `aumentarSalario($percentual)` que aplica o aumento
- Método `exibir()` que mostra nome, cargo e salário formatado
- Teste: crie um funcionário, aplique 15% de aumento e exiba antes e depois

Dica

Fórmula do aumento: `$this->salario = $this->salario * (1 + $percentual / 100)`; Para formatar: `number_format($this->salario, 2, ',', '.')`

```
<?php
```

```
class Funcionario {
```

```
    private $nome;  
    private $cargo;  
    private $salario;
```

```
    // Construtor
```

```
    public function __construct($nome, $cargo, $salario) {  
        $this->nome = $nome;
```

```

        $this->cargo = $cargo;
        $this->salario = $salario;
    }

    // Getters
    public function getNome() {
        return $this->nome;
    }

    public function getCargo() {
        return $this->cargo;
    }

    public function getSalario() {
        return $this->salario;
    }

    // Aplica aumento ao salário
    public function aumentarSalario($percentual) {
        $this->salario = $this->salario * (1 + $percentual / 100);
    }

    // Exibe os dados do funcionário
    public function exibir() {
        echo "Nome: " . $this->nome . "<br>";
        echo "Cargo: " . $this->cargo . "<br>";
        echo "Salário: R$ " . number_format($this->salario, 2, ',', '.') . "<br><br>";
    }
}

// Criação do funcionário
$funcionario = new Funcionario("Luiza", "Desenvolvedora", 3000);

// Exibe antes do aumento
echo "Antes do aumento:<br>";
$funcionario->exibir();

// Aplica aumento de 15%
$funcionario->aumentarSalario(15);

// Exibe depois do aumento
echo "Depois do aumento:<br>";

```

```
$funcionario->exibir();
```

```
?>
```

Foi criada a classe `Funcionario` com os atributos privados `$nome`, `$cargo` e `$salario`. O construtor recebe esses valores ao criar o objeto.

Os métodos getters permitem acessar os dados do funcionário. O método `aumentarSalario()` calcula o novo salário com base no percentual informado. Neste exemplo, foi aplicado um aumento de 15%.

O método `exibir()` mostra o nome, o cargo e o salário formatado utilizando `number_format()`, deixando o valor no padrão brasileiro.

No teste, foi criado um funcionário com salário de R\$ 3.000,00, exibido antes do aumento, aplicado um reajuste de 15% e exibido novamente. O novo salário passa a ser R\$ 3.450,00.

13 Classe Temperatura com conversão

Intermediário

Crie a classe Temperatura:

- Atributo privado: `$celsius`
- Construtor recebe o valor em Celsius
- Getter e setter para `$celsius`
- Método `paraFahrenheit()` — fórmula: $(C \times 9/5) + 32$
- Método `paraKelvin()` — fórmula: $C + 273.15$
- Exiba as três unidades para: 0°C, 100°C e 37°C

Dica

Os métodos de conversão apenas retornam o valor calculado — eles não alteram `$celsius`.
Use: `return ($this->celsius * 9/5) + 32;`

```
<?php
```

```
class Temperatura {
```

```
    private $celsius;
```

```
    // Construtor
```

```
    public function __construct($celsius) {  
        $this->celsius = $celsius;  
    }
```

```
    // Getter
```



```

public function getCelsius() {
    return $this->celsius;
}

// Setter
public function setCelsius($celsius) {
    $this->celsius = $celsius;
}

// Converte para Fahrenheit
public function paraFahrenheit() {
    return ($this->celsius * 9 / 5) + 32;
}

// Converte para Kelvin
public function paraKelvin() {
    return $this->celsius + 273.15;
}
}

// Temperatura 0°C
$temp1 = new Temperatura(0);

echo "Temperatura: " . $temp1->getCelsius() . "°C<br>";
echo "Fahrenheit: " . $temp1->paraFahrenheit() . "°F<br>";
echo "Kelvin: " . $temp1->paraKelvin() . " K<br><br>";

// Temperatura 100°C
$temp2 = new Temperatura(100);

echo "Temperatura: " . $temp2->getCelsius() . "°C<br>";
echo "Fahrenheit: " . $temp2->paraFahrenheit() . "°F<br>";
echo "Kelvin: " . $temp2->paraKelvin() . " K<br><br>";

// Temperatura 37°C
$temp3 = new Temperatura(37);

echo "Temperatura: " . $temp3->getCelsius() . "°C<br>";
echo "Fahrenheit: " . $temp3->paraFahrenheit() . "°F<br>";
echo "Kelvin: " . $temp3->paraKelvin() . " K<br>";

?>

```

Foi criada a classe Temperatura com o atributo privado \$celsius. O construtor recebe o valor da temperatura em Celsius quando o objeto é criado.

Os métodos getCelsius() e setCelsius() permitem acessar e alterar o valor da temperatura.

O método paraFahrenheit() converte a temperatura de Celsius para Fahrenheit usando a fórmula:

$$F=(C\times\frac{9}{5})+32$$

O método paraKelvin() converte a temperatura de Celsius para Kelvin usando a fórmula:

$$K=C+273.15$$

No teste foram criados três objetos com as temperaturas 0°C, 100°C e 37°C, exibindo seus valores em Celsius, Fahrenheit e Kelvin.

14 Classe Estoque com array de produtos

Intermediário

Crie a classe Estoque:

Atributo privado: \$produtos (array, inicia vazio)
Método adicionarProduto(\$nome, \$quantidade) que insere no array
Método listarProdutos() que exibe todos com um foreach
Método totalItens() que retorna a soma de todas as quantidades
Adicione 3 produtos e exiba o estoque completo com o total

Dica

Para adicionar: \$this->produtos[] = ['nome'=>\$nome,'qtd'=>\$quantidade]; No foreach: foreach(\$this->produtos as \$p){ echo \$p['nome'].': '.\$p['qtd']; }

```
<?php
```

```
class Estoque {
```

```
    private $produtos = [];
```

```
    // Adiciona um produto ao estoque
```

```
    public function adicionarProduto($nome, $quantidade) {
```

```
        $this->produtos[] = [
```

```
            'nome' => $nome,
```

```
            'qtd' => $quantidade
```

```
        ];
```

```

    }

    // Lista todos os produtos
    public function listarProdutos() {
        foreach ($this->produtos as $p) {
            echo $p['nome'] . ": " . $p['qtd'] . "<br>";
        }
    }

    // Calcula o total de itens
    public function totalItens() {
        $total = 0;

        foreach ($this->produtos as $p) {
            $total += $p['qtd'];
        }

        return $total;
    }
}

// Criação do estoque
$estoque = new Estoque();

// Adicionando produtos
$estoque->adicionarProduto("Caneta", 20);
$estoque->adicionarProduto("Caderno", 15);
$estoque->adicionarProduto("Lápis", 30);

// Exibindo os produtos
echo "Produtos em estoque:<br>";
$estoque->listarProdutos();

// Exibindo o total
echo "<br>Total de itens: " . $estoque->totalItens();

?>

```

Foi criada a classe Estoque, que possui o atributo privado \$produtos, iniciado como um array vazio.

O método adicionarProduto() adiciona um produto ao array, armazenando seu nome e quantidade.

O método listarProdutos() utiliza um foreach para percorrer o array e exibir todos os produtos cadastrados.

O método totalItens() também utiliza um foreach, somando as quantidades de todos os produtos e retornando o total.

No teste foram adicionados três produtos (Caneta

15	Classe Agenda de contatos (desafio)
-----------	--

Desafio

Una dois conceitos: use a classe Contato dentro de Agenda.

Classe Contato: atributos privados \$nome e \$telefone, construtor e getters

Classe Agenda: atributo privado \$contatos (array de objetos Contato)

Método adicionarContato(\$nome, \$telefone) que cria new Contato e insere no array

Método listar() que percorre o array e exibe cada contato

Adicione 3 contatos e liste a agenda

Dica

Em adicionarContato: \$c = new Contato(\$nome, \$telefone); \$this->contatos[] = \$c; Em listar: acesse com \$c->getNome() e \$c->getTelefone().

```
<?php

class Contato {

    private $nome;
    private $telefone;

    // Construtor
    public function __construct($nome, $telefone) {
        $this->nome = $nome;
        $this->telefone = $telefone;
    }

    // Getters
    public function getNome() {
        return $this->nome;
    }

    public function getTelefone() {
        return $this->telefone;
    }
}

class Agenda {

    private $contatos = [];

    // Adiciona um contato na agenda
```

```

    public function adicionarContato($nome, $telefone) {
        $c = new Contato($nome, $telefone);
        $this->contatos[] = $c;
    }

    // Lista os contatos
    public function listar() {
        foreach ($this->contatos as $c) {
            echo "Nome: " . $c->getNome() . "<br>";
            echo "Telefone: " . $c->getTelefone() . "<br><br>";
        }
    }
}

// Criação da agenda
$agenda = new Agenda();

// Adicionando contatos
$agenda->adicionarContato("Luiza", "559999999999");
$agenda->adicionarContato("Maria", "559888888888");
$agenda->adicionarContato("João", "559777777777");

// Exibindo os contatos
echo "Agenda de Contatos:<br><br>";
$agenda->listar();

?>

```

Foram criadas duas classes: **Contato** e **Agenda**.

A classe **Contato** possui os atributos privados **\$nome** e **\$telefone**, recebidos pelo construtor. Os métodos getters permitem acessar essas informações.

A classe **Agenda** possui o atributo **\$contatos**, que é um array de objetos da classe **Contato**. O método **adicionarContato()** cria um novo objeto **Contato** e o adiciona ao array. Já o método **listar()** percorre o array com um **foreach** e exibe os dados de cada contato utilizando os getters.

No teste, foram adicionados três contatos à agenda e, em seguida, todos foram listados na tela.

Bom trabalho! Em caso de dúvida, consulte o material da aula ou converse com os colegas.